Green Home Score & Retrofit Engine

How our system works, in plain English, and who builds what

What we are building

A web app where a homeowner can see how green their home is and what to do about it. It answers four questions:

- **How green is my home today?** A score from 0 to 100, with the reasons behind it.
- **What should I improve?** A short, ordered list of upgrades that suit this home.
- **How much energy and carbon could I save?** Electricity and gas shown separately.
- **Could my house be worth more?** A rough range, clearly marked as an estimate.

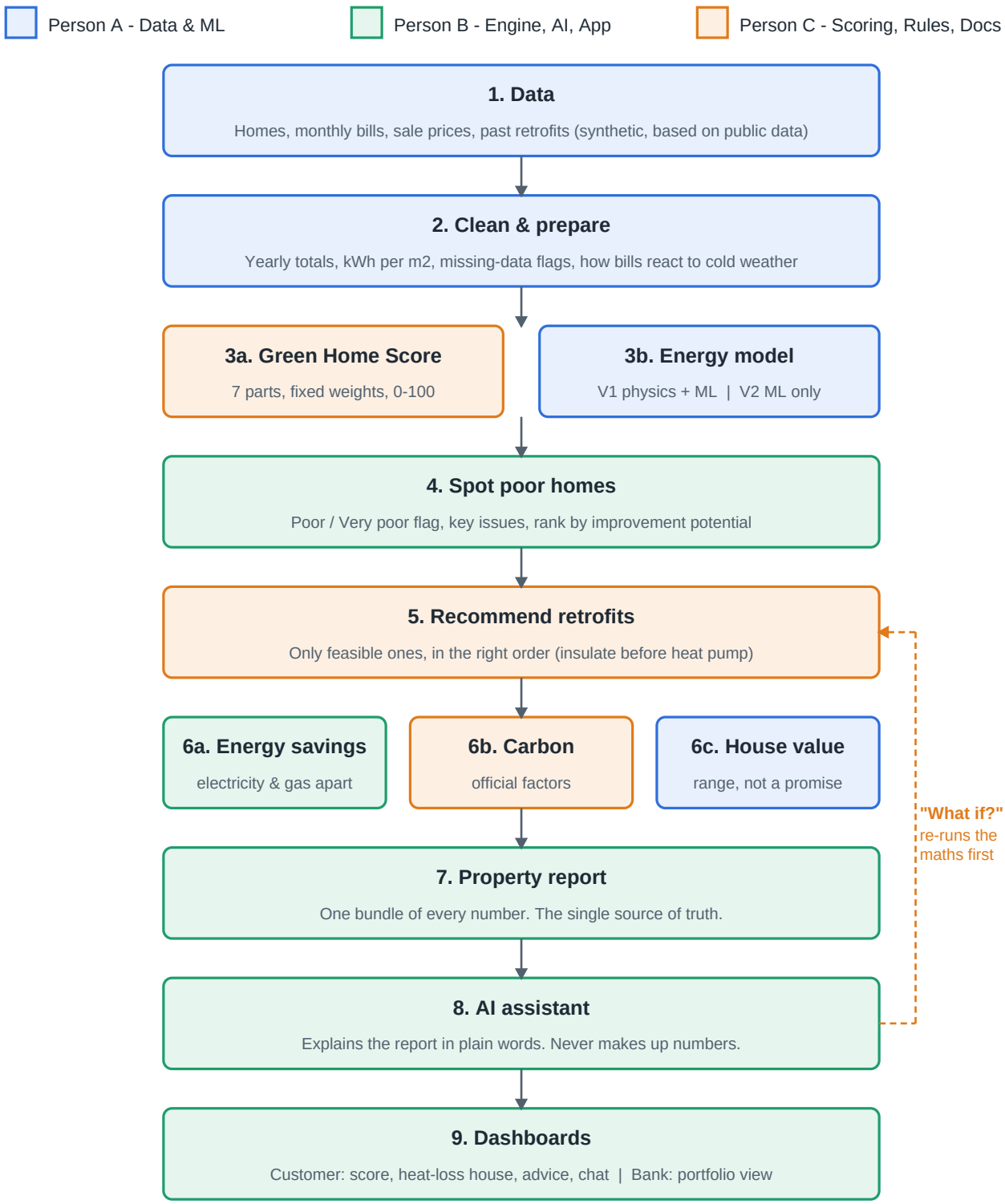
The five golden rules

The judges check these closely. Breaking any one of them costs marks.

#	Rule	What it means
1	**The AI never does the maths.**	Every number (score, savings, carbon, value) comes from our own code. The AI chat only explains those numbers. For a 'what if' question, the code reruns first and the AI explains the new result.
2	**Electricity and gas are always separate.**	A heat pump cuts gas but adds electricity. We must show both, not only the total.
3	**Every number has a source.**	Retrofit effects, costs and carbon factors all sit in config files with a note of where they came from.
4	**Be honest about uncertainty.**	Savings are 'estimates'. House value is an 'indicative range'. Missing data says 'unavailable', never 0.
5	**No cheating in testing.**	A home (and its owner) appears in only one of train / validation / test. Models never see information from the future.

How it flows

Data goes in at the top. The customer sees the dashboard at the bottom. Box colours show who builds each part.



Each step in plain words

1. The data

We cannot use real customer data, so we generate a realistic fake town of about 10,000-20,000 homes. The numbers are tuned to real public statistics: EPC certificates (home features and ratings), NEED (typical energy use) and Land Registry (house prices).

Trick that makes testing meaningful: every fake home also has **hidden traits** (how many people live there, how leaky it really is). These change its bills, but our models never see them, just like real life. Some homes also get a retrofit halfway through, so we can compare our predicted savings with what 'actually' happened.

2. Clean & prepare

Monthly bills become a yearly figure. Gas is mostly used in winter, so 3 summer months cannot simply be multiplied by 4. We weight each month by how much it normally counts. If too many months are missing, we say the figure is unreliable.

3a. The Green Home Score (0-100)

Seven parts, each scored 0-100, then combined with fixed weights set by the problem statement. A better home can never get a lower part-score.

Part	Weight	How we score it (draft)
Energy rating (EPC)	25%	A = 96, B = 86, C = 75, D = 61, E = 46, F = 30, G = 10
Energy use per m^2	20%	50 kWh/m^2 or less = 100, 350 or more = 0, straight line between
Insulation	15%	Loft and walls: none / partial / full
Heating	15%	Heat pump best; modern gas boiler middle; old boiler and direct electric low
Windows	10%	Triple 100, newer double 80, older double 60, partial 35, single 0
Solar / renewables	10%	Share of electricity the panels cover; none = 0
Smart controls	5%	None 0, timer 30, smart thermostat 70, smart + radiator valves 100

80-100	65-79	50-64	30-49	0-29
Highly efficient	Good	Moderate	Poor	Very poor

The dashboard shows how many points each part added, so the customer can see *why*.

3b. The energy model (we build two versions)

This model answers: how much electricity and gas should this home use, and what happens if we change it? Both versions plug into the same slot, so the rest of the system works with either.

Version 1: physics + ML (our main one). Work out how much heat leaks through each part of the house (roof, walls, windows, floor, draughts) using standard building tables. Multiply by how cold the region's year was ("degree days"). That gives the heating needed.

Then we **tune it to the home's own bills**: compare monthly gas with monthly cold to see how strongly this home actually reacts to cold. If it uses 30% more than its walls and roof suggest, its tuning factor is 1.3 (more people at home, warmer thermostat, hidden draughts). A small ML model then fixes whatever is left over.

Version 2: ML only. LightGBM learns energy use straight from home features. We add rules so that, for example, more insulation can never predict *more* gas. We also report a third, physics-only version as a simple baseline.

> **Why physics?** A retrofit becomes a real change to the house: loft insulation makes the roof leak less, and we run the model again. Topping up existing insulation automatically saves less than insulating a bare loft. Insulate-then-heat-pump interactions come out naturally. And we can tell the customer "your roof is 29% of your heat loss".

Keeping the test fair: our fake data uses hidden "true" leakiness and habits for each home, while the model only sees the standard tables. Otherwise the physics would look perfect just because it matches how we made the data. Because it knows what similar homes should use, the model also stops big houses being flagged just for being big.

4. Spot poor homes

Homes scoring Poor or Very poor are flagged with their main problems, for example "single glazing" or "old boiler". We rank them by **improvement potential**: the % cut in bought energy (grid electricity + gas) if they did every suitable upgrade.

5. Recommend retrofits

A rulebook decides what is **possible** for each home. Examples: no cavity-wall insulation on solid walls, no new double glazing if it is already double glazed, solar only if there is a usable roof. Each suggestion gets High / Medium / Low suitability and a one-line reason. Order matters: fix the walls and roof first, then controls, then heating, then solar.

6a. Energy savings

We apply upgrades one at a time, in order (fabric, then controls, then heating, then solar). Each upgrade changes the house, and the energy model runs again, so every step builds on the last. This avoids double-counting: once the loft is insulated, the heat pump only has to cover the smaller heat demand that is left.

A heat pump makes gas go down and electricity go up, and we show the electricity increase as a negative saving. Solar lowers what you buy from the grid, not what the home uses. We test both energy versions against homes in our data that really were retrofitted, and compare them.

6b. Carbon

Carbon saved = electricity saved x electricity factor + gas saved x gas factor. Factors come from the UK government's official yearly table (DESNZ). Roughly 0.21 kg CO_2e per kWh for electricity and 0.18 for gas; to be confirmed against the official file.

6c. House value

A model learns house prices from sales data. We ask it the price of the same house, on the same date, before and after the upgrades, and report the difference as a **range** (e.g. +2% to +5%). We always say it is an indicative estimate, not a guarantee.

7-9. Report, AI assistant, dashboard

All results go into one **property report**. The AI assistant can only read that report or ask the code to rerun with different upgrades. A checker then scans every AI answer: any number not in the report is flagged and removed. The dashboard shows the score breakdown, recommendations, savings bars, carbon, value range and a chat box.

What makes us stand out

Heat-loss house (customer view)

A drawing of the home with the roof, walls, windows and floor coloured by how much heat each one leaks. Click a part to see the fix and its saving. Toggle an upgrade and the house recolours, the score moves and the savings update, because the code reruns. It is a 'what if' you can see, not just type.

Lender portfolio view (bank view)

Lloyds is a bank, so we add a page for the bank, not just the homeowner. It covers the whole (fake) mortgage book: the mix of scores and ratings, how many homes are Poor, total carbon linked to the loans ("financed emissions"), how much that drops if the worst homes upgrade, and a ranked list of customers to offer green finance to. It reuses the same reports, so no new models are needed.

Memory that learns

The app remembers each customer, for example "I rent, I can't change the boiler", "budget £5k" or "already did the loft". Finished upgrades update the house, limits become rules, and rejected ideas move down the list with a note saying why. A timeline shows the advice changing over three chats. The numbers still come from the code.

Fairness check

We check whether results are unfair to some groups (regions, flats, very old homes, cheaper homes), for example whether the energy model is less accurate for them. Homes on a tight budget see cheap and grant-funded options first. Because the data is fake, this tests how our system behaves, not real-world fairness, and we say so.

Who does what

Persons A and B have stronger AI coding tools, so they take the parts that need a lot of code. Person C takes the parts that need careful thinking, research and checking by hand. Those parts are small in code but worth a lot of marks: the score alone is 15%.

Person A - Data & Machine Learning

Lots of code: data generation and model training.

- Build the fake-town data generator (homes, bills, sales, past retrofits, hidden traits, some missing data on purpose)
- Yearly totals from monthly bills; split data into train / validation / test by customer
- Energy model in two versions: physics + ML and ML only (plus physics-only baseline), tuned to each home's bills
- House-value model, compared against a simple baseline
- Model test results and final test predictions (submission items G and I)
- Write DATA.md: where data came from, how labels were made, limitations

Person B - Engine, AI assistant & App

Lots of code: wiring everything together, plus the AI and the UI.

- Day 1, hour 1: write the shared data definitions (schema.py) so everyone codes against them
- Savings calculator (electricity and gas, upgrades applied in order) and the 'what if' path
- AI assistant with tools and the number-checker; run the AI tests
- Backend API and the customer dashboard with the heat-loss house
- Lender portfolio view and memory that learns
- Write LLM.md and draw the architecture diagram (items E and F)

Person C - Scoring, Rules & Docs

Small, careful, easy-to-check pieces. Light AI use is enough.

- Green Home Score: the 7 scoring tables in a config file, the scoring code, tests that weights add to 1 and better homes never score lower
- Retrofit rulebook: when each upgrade is possible, cost range, typical effect, with sources
- Carbon factors from DESNZ and the carbon calculation
- Physics tables with sources: heat-loss values, house sizes, regional degree days, boiler/heat-pump efficiency, grant rules
- Fairness check and its write-up
- Write ~50 tricky test questions for the AI (missing data, impossible upgrades, heat pump raising electricity)
- Write SCORING.md and the Responsible AI note (items D and H); keep the final checklist

Tip: when Person C needs boilerplate code (file loaders, test scaffolding), A or B can generate it quickly with their tools. C writes the spec, they produce the code.

Timeline & hand-offs

When	What gets handed over	From -> To
Hour 1	Shared data definitions agreed; everyone can start	B -> all
~Hour 4	First rough dataset	A -> B, C
~Hour 6	Retrofit rulebook, carbon factors, physics tables	C -> A, B
~Hour 8	Energy model, both versions (first cut)	A -> B
~Hour 9	Score calculation working	C -> B
~Hour 12	House-value model ready	A -> B
~Hour 14	One home works end to end, from data to dashboard	B
~Hour 18	Heat-loss house, portfolio view, memory	B
After that	AI tests, V1 vs V2 comparison, fairness check; results written up	All

What we must hand in

Item	Deliverable	Owner
A	Working web app / dashboard	B
B	Code and data scripts, with run instructions	All
C	Trained models and the code that uses them	A
D	Scoring method document	C
E	AI details: prompts, model, how we tested it	B
F	Architecture diagram	B
G	Evaluation report (validation results vs simple baselines)	A + B
H	Responsible AI note	C
I	Held-out test predictions and data documentation	A

Full technical detail (interfaces, file layout, formulas) lives in docs/ARCHITECTURE.md.